

**Procesamiento de Lenguaje Natural con
Aprendizaje Profundo,
Máster en Ciencia y Tecnología Informática**

**Tema 6 Data Augmentation
Curso 2025-2026**

Objetivos

- Comprender el concepto de Data Augmentation (DA).
- Comprender y aplicar los principales enfoques de DA.
- Conocer y aplicar las principales librerías que implementan las técnicas de DA para PLN.

Índice

- ¿Qué es Data Augmentation (DA)?
- DA en visión artificial
- DA en PLN
- Principales enfoques / librerías de DA para PLN

¿Por qué Data Augmentation?

- Algoritmos supervisados requieren **grandes colecciones de datos anotados** para **obtener buenos resultados**.
- Proceso de **anotación**: costoso y lento.
 - Varios anotadores (expertos).
 - Crear guías de anotación.
 - Medir el acuerdo entre anotadores.

Data Augmentation (DA)

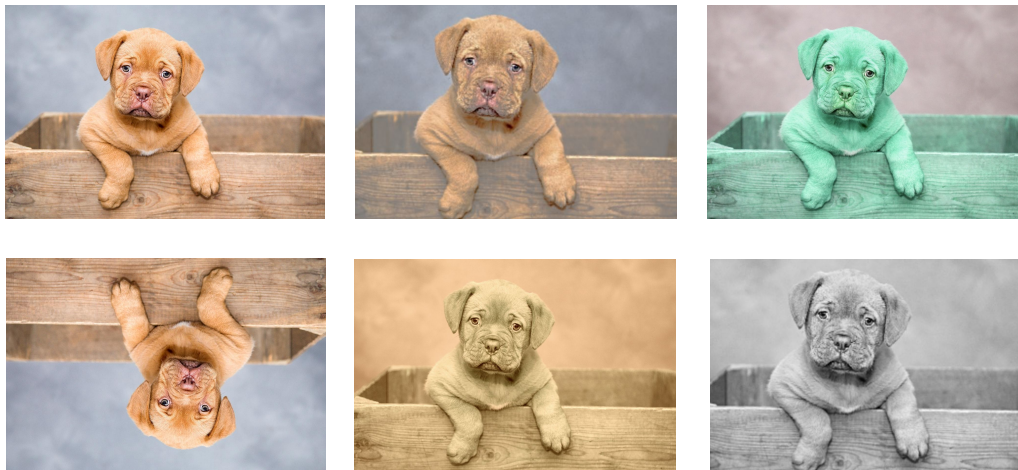
- **DA** son técnicas dirigidas a **generar nuevos datos sintéticos** a partir de los datos disponibles.
- Cómo? **modificaciones** sobre los **datos originales**.
- Aumentan tamaño dataset, y añaden diversidad => evitan sobre-aprendizaje.

Índice

- ¿Qué es Data Augmentation (DA)?
- **DA en visión artificial**
- DA en PLN
- Principales técnicas / librerías de DA para PLN

DA en visión artificial

- En este campo, las modificaciones más habituales son cambios en tamaño, color, contraste, brillo, rotaciones, zoom, etc.
- [albumentations](#) es una librería de Python para generar nuevas imágenes.



Índice

- ¿Qué es Data Augmentation (DA)?
- DA en visión artificial
- **DA en PLN**
- Principales técnicas / librerías de DA para PLN

¿Cómo generar textos sintéticos?

DA en PLN

- **Añadir, eliminar o intercambiar caracteres, tokens, y/u oraciones.**
- Principales enfoques (librerías) DA para PLN:
 - [NLPAug.](#)
 - [Easy Data Augmentation \(EDA\).](#)
 - [An Easier Data Augmentation \(AEDA\)](#)
 - Back translation.
 - LLMs

NLPAug

- En tres **niveles**: **carácter**, **palabra** u **oración**.
- Para cada nivel:
 - eliminación aleatoria,
 - inserción aleatoria,
 - alteración del orden,
 - **sustitución de sinónimos**.

NLPAug: reemplazo de caracteres

- Además del reemplazo aleatorio, NLPAug incluye otros métodos para reemplazar caracteres por otros, como
 - **Optical character recognition** o
 - **Keyboard Augmenter**.

NLPAug: Optical character recognition (OCR)

- Genera nuevas instancias simulando errores basados en que algunos caracteres tienen grafía similar.
- Útil en aplicaciones donde el objetivo es reconocer texto en imágenes, y transformar imágenes en texto (por ejemplo, digitalizar documentos)

El barco estaba lleno de peces



***B**l barco e**8**ta**6**a **11**eno de peces*

E -> B

s -> 8

b -> 6

l -> 1

NLPAug: Optical character recognition (OCR)

```
1 import nlpaug.augmenter.char as nac
2 text = 'El Parlamento insta a prohibir el cobro por el equipaje de mano, pero aún lo seguirás pagando.'
3
4 aug = nac.OcrAug()
5 augmented_texts = aug.augment(text, n=3)
6
7 print("Texto original:")
8 print(text)
9 print()
10 print("Textos generados:")
11 for new_text in augmented_texts:
12     print(new_text)
```

Texto original:
El Parlamento insta a prohibir el cobro por el equipaje de mano, pero aún lo seguirás pagando.

Textos generados:

El Parlamento insta a prohibir el cobro por el equipaje de mano, peku aún 10 seguirás pagando.

El Parlamento insta a pkohi6ik el cobro p0k el equipaje de manu, pero aún 10 seguirás pa9and0.

El Bukuparlamentu insta a pr0hi6ik el cobro puk el equipaje de mano, pero aún lo seguirás pa9andu.

NLPAug: Keyboard Augmenter

- Genera nuevas instancias reemplazando algunos caracteres por otros que están próximos en el teclado

El barco estaba lleno de peces



*El bar**vp** estaba **ñ**leno de **oe**ves*

c -> v
o -> p
l -> ñ
p -> o

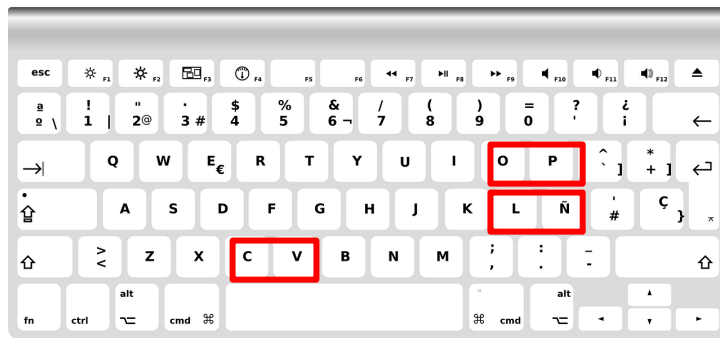


Imagen de [OpenClipart-Vectors](#) en [Pixabay](#)

NLPAug: Keyboard Augmenter

- **Keyboard Augmenter:** genera nuevas instancias reemplazando algunos caracteres por otros que están próximos en el teclado

```
1 aug = nac.KeyboardAug()  
2 augmented_texts = aug.augment(text, n=3)  
3 print("Texto original:")  
4 print(text)  
5 print()  
6 print("Textos generados:")  
7 for new_text in augmented_texts:  
8     print(new_text)
```



Texto original:

El Parlamento insta a prohibir el cobro por el equipaje de mano, pero aún lo seguirás pagando.

Textos generados:

El WuroLa5laJentL insta a prohibir el cob#9 por el eAu7laje de mano, p3r) aún lo eWguieás 0zganWo.

El EKGopsFlamen\$o insta a (rohigKr el coVrl por el equil)ajD de HaMo, pero aún lo serHitás pagando.

El Parlamento 7nsfa a proM&bor el vobrp por el #qu7pxje de Hsno, pero aún lo s#gu9ráa pagando.

NLPAug: Reemplazo de sinónimos

- Basado en el uso de diccionarios (como WordNet), word embeddings (Word2Vec) o LLMs (BERT, RoBERT, GPT, etc)

*El **barco** estaba lleno de **peces***



*El **navío** estaba lleno de **pescados***

NLPAug: Reemplazo de sinónimos

- Aunque conserva significado, no está exento de errores:

El barco estaba lleno de peces



*El **embarcación** estaba lleno de **pescados***

No hay concordancia en género

NLPAug: Reemplazo de sinónimos

```
[ ] 1 aug = naw.SynonymAug(aug_src='wordnet')
    2 text = 'The key opens all the locks in the house'
    3 augmented_texts = aug.augment(text, n=3)
    4
    5 print("Texto original:")
    6 print(text)
    7 print()
    8 print("Textos generados:")
    9 for new_text in augmented_texts:
10 |     print(new_text)
```

Texto original:

The key opens all the locks in the house

Textos generados:

The key give all the locks in the sign

The cardinal opens totally the locks in the theater

The key opens wholly the locks in the household

NLPAug: Reemplazo de sinónimos

```
2 aug = naw.ContextualWordEmbsAug(model_path='bert-base-uncased', action="substitute")
3 augmented_texts = aug.augment(text, n=3)
4
5 print("Texto original:")
6 print(text)
7 print()
8 print("Textos generados:")
9 for new_text in augmented_texts:
10     print(new_text)
```

Texto original:

The key opens all the locks in the house

Textos generados:

a key opened all the locks to the house

the car opens all 42 doors in the house

the key opens any possible locks to the house

NLPAug: Reemplazo por antónimos

```
1 aug = naw.AntonymAug()  
2 text = 'A Good Life offers a very different perspective'  
3 augmented_texts = aug.augment(text, n=3)  
4  
5 print("Texto original:")  
6 print(text)  
7 print()  
8 print("Textos generados:")  
9 for new_text in augmented_texts:  
10 |     print(new_text)
```

Texto original:

A Good Life offers a very different perspective

Textos generados:

A Bad Life offers a very like perspective

A Evil Life offers a very same perspective

A Evil Life offers a very same perspective

DA con NLPAug

Ventajas

- Soporta múltiples niveles: carácter, palabra y oración
- Gran variedad de técnicas (sinónimos, inserción, OCR, embeddings, etc.)
- Flexible y configurable
- Útil para distintos tipos de tareas en NLP

Limitaciones

- Puede generar textos **poco naturales (con errores gramaticales o semánticos)**
- No todas las transformaciones son adecuadas para todas las tareas

Easy Data Augmentation (EDA)

- Librería TextAugment
- Operaciones muy básicas:
 - intercambio aleatorio.
 - borrado aleatorio.
 - reemplazo de sinónimos.
 - inserción de sinónimos.

EDA: intercambio aleatoria

- En esta operación, se seleccionan **aleatoriamente dos palabras**, y se **intercambian** sus posiciones.
- Puede producir errores gramaticales y semánticos

*This **article** will focus on summarizing data augmentation **techniques** in NLP*



*This **techniques** will focus on summarizing data augmentation **article** in NLP*

EDA: borrado aleatorio

- Se selecciona una palabra aleatoriamente y se elimina del texto. Puede ejecutarse varias veces.
- Puede producir errores gramaticales y semánticos

*This **article** will focus on summarizing data augmentation techniques in NLP*



*This will focus on summarizing data augmentation **techniques** in NLP*



This will focus on summarizing data augmentation in NLP

EDA: reemplazo e inserción de sinónimos

- Utiliza la librería **nlk** para
 - los **sinónimos** de [WordNet](#) o de su versión multilingüe [OMW](#).
 - **obtener** la lista de **stopwords** (sobre este tipo de palabras no se aplica ninguna operación).

EDA: reemplazo de sinónimos

- En esta operación, se seleccionan **n palabras** (que no sean stopwords) en un texto para ser **reemplazadas** por **sinónimos** para generar un nuevo texto.

*This **article** will focus on summarizing data augmentation **techniques** in NLP*



*This **write-up** will focus on summarizing data augmentation **methods** in NLP*

EDA: inserción aleatoria de sinónimos

- En esta operación, se selecciona una **palabra** (que no sea una stopword) de forma **aleatoria**, e **inserta** un **sinónimo** de dicha palabra en una **posición aleatoria** en la oración. Pueden producir errores gramaticales y semánticos
- Este proceso se realiza n veces.
- Las palabras originales no son eliminadas o reemplazadas. Por ejemplo:

*This **article** will focus on summarizing data augmentation techniques in NLP*



*This article will focus on **write-up** summarizing data augmentation **techniques** in NLP*



*This article will focus on **write-up** summarizing data augmentation techniques in NLP **methods***

DA con EDA

Ventajas

- Técnica **simple y rápida** (no requiere modelo externos; los **sinónimos los obtiene de WordNet**).
- Útil en tareas de **clasificación de textos**

Limitaciones

- Puede generar textos **poco naturales** (introduce errores **gramaticales**).
- Puede modificar el **significado original**
- No adecuada para tareas complejas (resumen, traducción)

AEDA: An Easier Data Augmentation

- Implementada en TextAugment.
- Técnica muy simple: inserción aleatoria de signos de puntuación (. , ! ? ; :)
- Mejora la robustez del modelo

This article will focus on summarizing data augmentation techniques in NLP



This article , will focus on summarizing data ! augmentation techniques in NLP

DA con AEDA

Ventajas

- Muy **simple y eficiente**
- No modifica palabras → preserva mejor el significado
- Introduce **ruido ligero**
- Útil en tareas de **clasificación de textos**

Limitaciones

- No introduce gran diversidad lingüística
- No adecuada para tareas complejas (resúmenes, traducción).

Ejemplos prácticos de NLPAug, EDA y AEDA

- [NLPAug](#)
- [TextAugmenter \(EDA\)](#)
- [TextAugmenter \(AEDA\)](#)

DA con Back translation

- Traducir el texto original a un idioma, y volver a traducirlo al idioma original.

No me esperes para el almuerzo



Don't wait me for lunch



No me esperes para almorzar



Imagen de [Raphael Silva](#) en [Pixabay](#)



Imagen de [OpenClipart-Vectors](#) en [Pixabay](#)



Imagen de [Raphael Silva](#) en [Pixabay](#)

DA con Back translation

Ventajas

- Genera **textos naturales** (paráfrasis realistas)
- Mantiene, en general, el **significado original**
- Introduce **variabilidad lingüística**
- Mejora la **generalización del modelo**

Limitaciones

- **Coste computacional** elevado.
- Depende de la **calidad del traductor**.
- Puede distorsionar el significado (*El paciente mejoró después del tratamiento -> El paciente cambió después del tratamiento*).

DA con LLMs

- Utiliza un modelo LLM para obtener una nueva versión del texto (paráfrasis) conservando su significado (y su label)

El servicio fue muy lento



La atención tardó bastante

DA con LLMs

Ventajas

- Genera **textos naturales y coherentes**
- Mantiene mejor el **significado original**
- Introduce gran **diversidad lingüística**
- Adaptable a distintas tareas (clasificación, QA, etc.)

Limitaciones

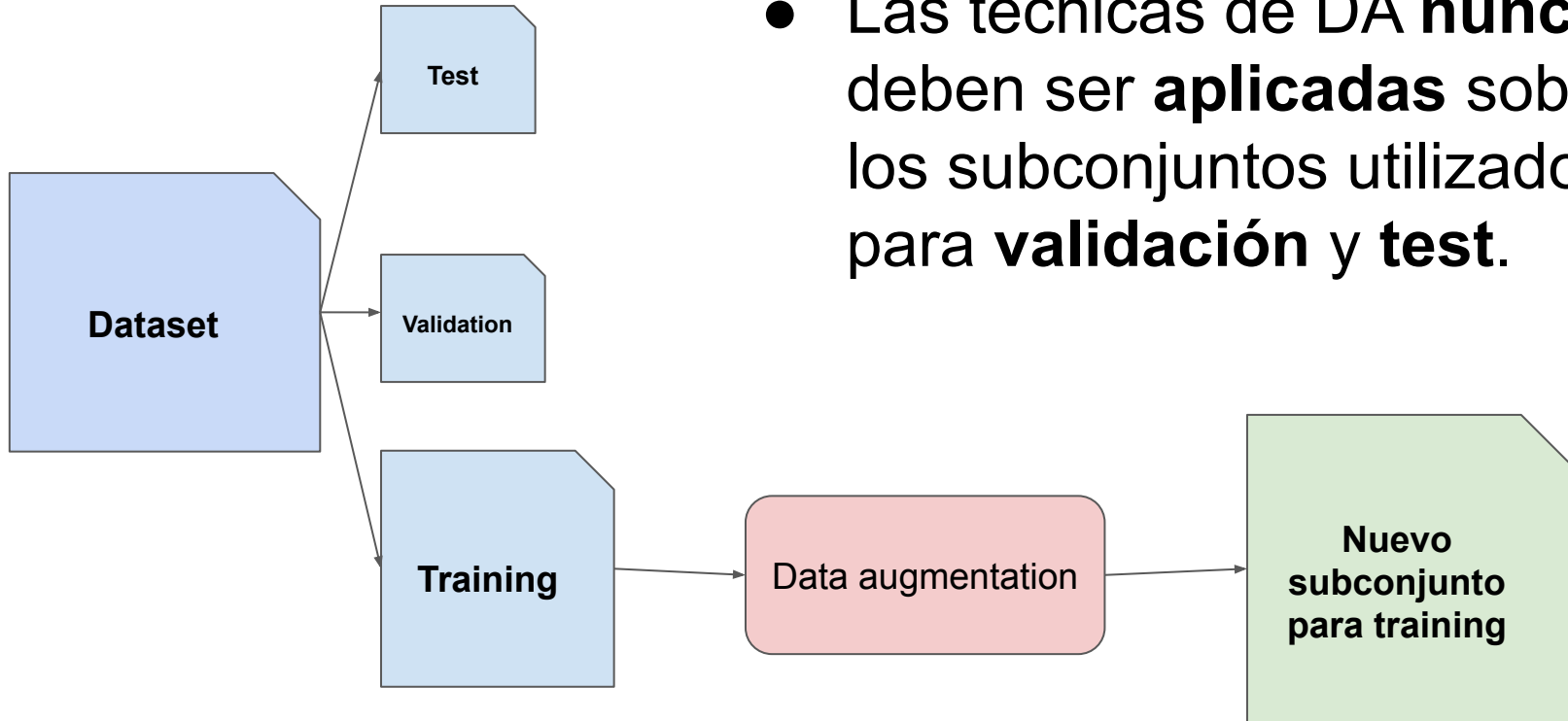
- **Coste computacional** elevado
- Dependencia de modelos externos
- Posibles sesgos, **alucinaciones** o cambios de significado

Ejemplos prácticos de DA con back translation y LLMs

- [DA con back translation.](#)
- [DA con LLMs.](#)

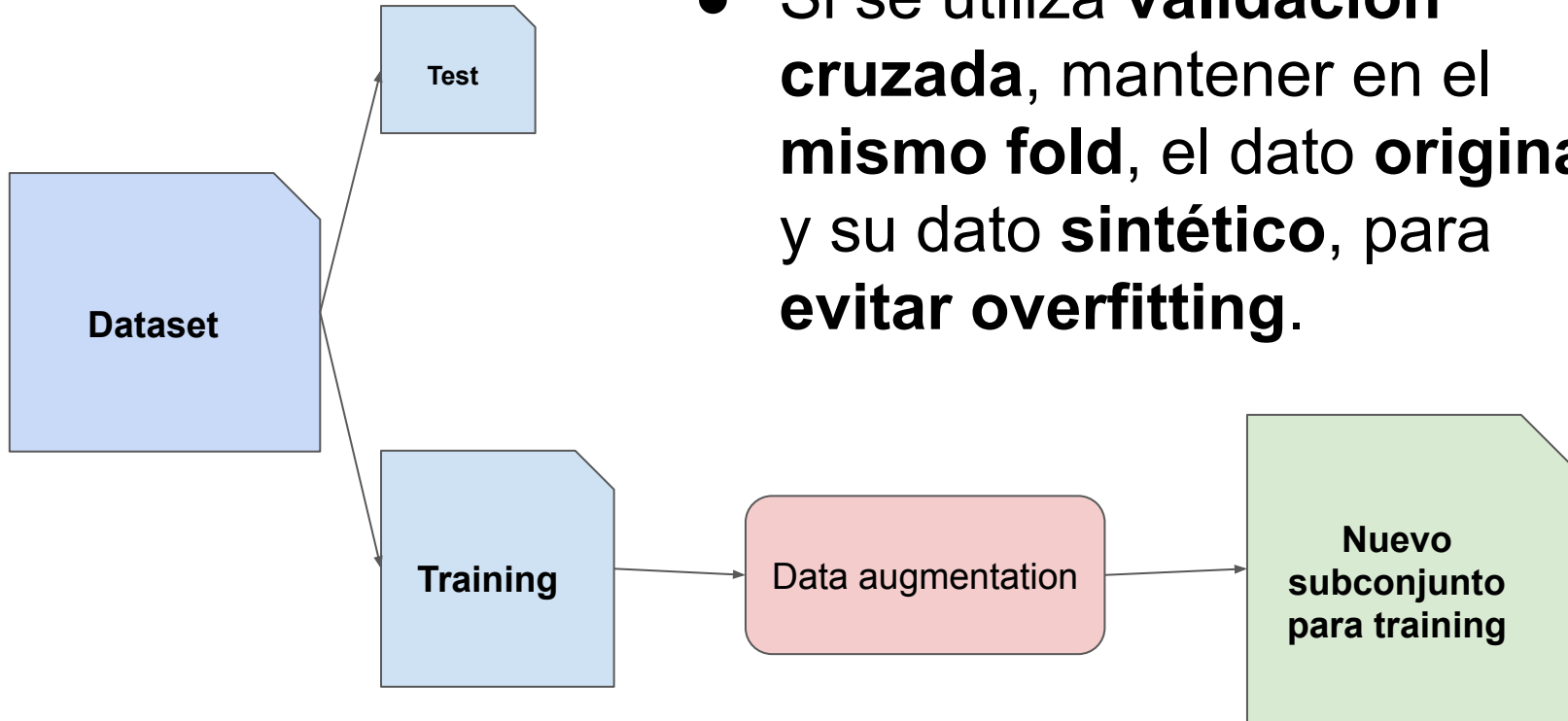
DA en PLN

- Las técnicas de DA **nunca** deben ser **aplicadas** sobre los subconjuntos utilizados para **validación** y **test**.



DA en PLN

- Si se utiliza **validación cruzada**, mantener en el **mismo fold**, el dato **original** y su dato **sintético**, para **evitar overfitting**.

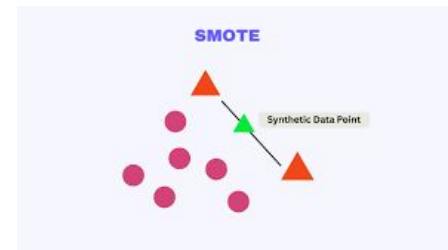


Resumen

- Aprendizaje supervisado necesita grandes cantidades de datos anotados.
- En PLN, DA genera textos sintéticos (modificaciones sobre los originales).
- Ojo!!! Los nuevos textos sintéticos pueden contener errores gramaticales, e incluso modificar su significado original.
- Estudio empírico para determinar qué técnicas y qué número de ejemplos sintéticos.
- Test y validación no deben incluir datos sintéticos.
- No siempre ayudan a mejorar los resultados en PLN.

SMOTE en PLN, ¿sí o no?

- Synthetic Minority Over-sampling Technique
- Técnica que permite generar ejemplos en las clases minoritarias.
- Se aplica sobre vectores numéricos
- Genera nuevos datos interpolando entre ejemplos.



⚠ Limitaciones en PLN + deep learning

- Genera datos en el **espacio vectorial**, no genera texto real.
- Puede producir ejemplos **poco realistas** (representaciones sin significado) y afectar al aprendizaje

[Ejemplo PLN con SMOTE](#)

OpenCourseWare
Procesamiento de Lenguaje Natural con
Aprendizaje Profundo,

Gracias!!!

<https://github.com/iseaura>